Detailed Notes on OOP Lecture 6: Generics, Custom ArrayList, Lambda Expressions, Exception Handling, Object Cloning

This document provides comprehensive notes for the sixth video of the Community Classroom's Object-Oriented Programming (OOP) course in Java, focusing on generics, creating a custom ArrayList, lambda expressions, exception handling, and object cloning. The notes are organized by the topics covered in the video, including key concepts, code snippets, and examples, with additional explanations to ensure clarity and retention for aspiring elite coders aiming for high-paying tech roles.

1 Generics in Java

1.1 Key Concepts

- **Generics**: Introduced in Java 5, generics enable type-safe collections and classes by allowing parameterization of types.
- **Purpose**: Eliminates the need for casting, improves code readability, and catches type-related errors at compile time.
- **Syntax**: Use angle brackets `<T>` to define a generic type (e.g., `ArrayList<String>`).
- **Key Features**:
  - `T` (Type Parameter): Represents any reference type.
  - Bounded Types: Restrict types (e.g., `<T extends Number>`).
  - Wildcards: `<? extends Type>` (upper bound), `<? super Type>` (lower bound), `<?>` (unbounded).
- **Benefits**: Type safety, code reusability, and reduced runtime errors.

1.2 Explanation

The video explains that generics prevent runtime errors by ensuring type consistency at compile time. It contrasts non-generic collections (requiring casting) with generic ones, emphasizing type safety.

1.3 Example from Video

The video demonstrates a non-generic `ArrayList` causing runtime issues and a generic `ArrayList<String>` ensuring type safety.

1.4 Code Snippet

```java
// Non-generic ArrayList (pre-Java 5)
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList list = new ArrayList();
        list.add("Hello");
        list.add(42); // No compile-time error
        String s = (String) list.get(1); // Runtime error:
            ClassCastException
    }
}

// Generic ArrayList
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Hello");
        // list.add(42); // Compile-time error
        String s = list.get(0); // No casting needed
        System.out.println(s); // Output: Hello
    }
}
```

Important Points:

- Generics ensure type safety, preventing `ClassCastException`.
- Use bounded types for specific constraints (e.g., `<T extends Number>` for numeric types).
- Wildcards provide flexibility in method parameters but restrict modifications.

1.5 Additional Example (Bounded Types)

```java
class Box<T extends Number> {
    private T value;

    public void setValue(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
```

```
    }
}

public class Main {
    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>();
        intBox.setValue(42);
        System.out.println(intBox.getValue()); // Output: 42
        // Box<String> strBox = new Box<>(); // Compile-time
            error
    }
}
```

2 Creating a Custom ArrayList

2.1 Key Concepts

- **Custom ArrayList**: A user-defined dynamic array implementation using generics for type safety.
- **Key Components**:
  - Internal array to store elements.
  - Methods like `add`, `remove`, `get`, and `size`.
  - Dynamic resizing when the array is full.
- **Purpose**: Understand how collections like `ArrayList` work internally, a common interview topic.

2.2 Explanation

The video walks through building a simplified generic `MyArrayList` class, demonstrating dynamic resizing and basic operations. It emphasizes the importance of generics for type safety and array resizing for scalability.

2.3 Example from Video

The video creates a `MyArrayList` class to store integers, showing how to add elements and resize the internal array.

2.4 Code Snippet

```
public class MyArrayList<T> {
    private Object[] data;
    private int size;
    private int capacity;

    public MyArrayList() {
        capacity = 10;
        data = new Object[capacity];
```

```
        size = 0;
    }

    public void add(T element) {
        if (size == capacity) {
            // Resize array
            capacity *= 2;
            Object[] newData = new Object[capacity];
            for (int i = 0; i < size; i++) {
                newData[i] = data[i];
            }
            data = newData;
        }
        data[size++] = element;
    }

    public T get(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException();
        }
        return (T) data[index];
    }

    public int size() {
        return size;
    }
}

public class Main {
    public static void main(String[] args) {
        MyArrayList<String> list = new MyArrayList<>();
        list.add("Apple");
        list.add("Banana");
        System.out.println("Size: " + list.size()); // Output:
            Size: 2
        System.out.println("Element at 0: " + list.get(0)); //
            Output: Element at 0: Apple
    }
}
```

Important Points:

- Dynamic resizing doubles the array size when full, balancing performance and memory.
- Generics ensure type safety for the custom `ArrayList`.
- Understanding internal implementations is crucial for system design interviews.

3 Lambda Expressions

3.1 Key Concepts

- **Lambda Expressions**: Introduced in Java 8, they provide a concise way to represent anonymous functions (functional interfaces).
- **Functional Interface**: An interface with a single abstract method (SAM), marked with `@FunctionalInterface`.
- **Syntax**: `(parameters) -> expression` or `(parameters) -> { statements; }`.
- **Purpose**: Simplifies code for functional programming, used extensively in streams and collections.

3.2 Explanation

The video explains that lambda expressions replace verbose anonymous classes for functional interfaces, making code cleaner. It highlights their use with the `java.util.function` package (e.g., `Consumer`, `Function`).

3.3 Example from Video

The video shows a lambda expression implementing a `Runnable` interface for a thread.

3.4 Code Snippet

```
// Without lambda
public class Main {
    public static void main(String[] args) {
        Runnable r = new Runnable() {
            @Override
            public void run() {
                System.out.println("Thread running...");
            }
        };
        new Thread(r).start();
    }
}

// With lambda
public class Main {
    public static void main(String[] args) {
        Runnable r = () -> System.out.println("Thread running...");
        new Thread(r).start(); // Output: Thread running...
    }
}
```

3.5 Additional Example (Functional Interface)

```java
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}

public class Main {
    public static void main(String[] args) {
        Calculator add = (a, b) -> a + b;
        Calculator multiply = (a, b) -> a * b;
        System.out.println("Add: " + add.calculate(5, 3)); // Output: Add: 8
        System.out.println("Multiply: " + multiply.calculate(5, 3)); // Output: Multiply: 15
    }
}
```

Important Points:

- Lambda expressions are ideal for functional interfaces, reducing boilerplate code.
- They enable functional programming in Java, used in APIs like Streams.
- Common in modern Java frameworks (e.g., Spring Reactor, JavaFX).

4 Exception Handling

4.1 Key Concepts

- **Exception**: An event that disrupts normal program flow (e.g., `NullPointerException`).
- **Types**:
  - **Checked Exceptions**: Must be handled or declared (e.g., `IOException`).
  - **Unchecked Exceptions**: Optional to handle (e.g., `RuntimeException`).
- **Keywords**:
  - `try`: Block for code that might throw an exception.
  - `catch`: Handles specific exceptions.
  - `finally`: Executes regardless of exception (e.g., for cleanup).
  - `throw`: Throws an exception explicitly.
  - `throws`: Declares exceptions a method might throw.
- **Custom Exceptions**: User-defined exceptions extending `Exception` or `RuntimeException`.

4.2 Explanation

The video stresses the importance of exception handling for robust applications, demonstrating try-catch blocks and custom exceptions. It explains the hierarchy: `Throwable` `Error`, `Exception`.

4.3 Example from Video

The video shows handling an `ArithmeticException` for division by zero.

4.4 Code Snippet

```java
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero");
        } finally {
            System.out.println("Cleanup done");
        }
    }
}
// Output:
// Error: Division by zero
// Cleanup done
```

4.5 Custom Exception Example

```java
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class Main {
    public static void setAge(int age) throws InvalidAgeException
        {
        if (age < 0 || age > 150) {
            throw new InvalidAgeException("Invalid age: " + age);
        }
        System.out.println("Age set to: " + age);
    }

    public static void main(String[] args) {
        try {
            setAge(-5);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage()); // Output:
               Invalid age: -5
        }
    }
}
```

Important Points:

- Use checked exceptions for recoverable errors, unchecked for programming errors.

- `finally` ensures cleanup (e.g., closing resources).
- Custom exceptions improve code clarity and error handling specificity.

5 Object Cloning

5.1 Key Concepts

- **Object Cloning**: Creating a copy of an object using the `Cloneable` interface and `clone()` method.
- **Types**:
  - **Shallow Copy**: Copies fields, but references point to the same objects.
  - **Deep Copy**: Copies fields and creates new instances for referenced objects.
- **Key Steps**:
  - Implement `Cloneable`.
  - Override `clone()` from `Object`.
- **Purpose**: Create independent copies of objects to avoid unintended modifications.

5.2 Explanation

The video explains that cloning is useful when you need a copy of an object without affecting the original. It contrasts shallow and deep copying, highlighting the need for deep copies with complex objects.

5.3 Example from Video

The video demonstrates shallow cloning with a `Person` class containing a reference to an `Address` object.

5.4 Code Snippet (Shallow Copy)

```
class Address {
    String city;

    Address(String city) {
        this.city = city;
    }
}

class Person implements Cloneable {
    String name;
    Address address;

    Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }
```

```
17
18      @Override
19      protected Object clone() throws CloneNotSupportedException {
20          return super.clone();
21      }
22  }
23
24  public class Main {
25      public static void main(String[] args) throws
            CloneNotSupportedException {
26          Address addr = new Address("Delhi");
27          Person p1 = new Person("Alice", addr);
28          Person p2 = (Person) p1.clone();
29
30          p2.name = "Bob";
31          p2.address.city = "Mumbai";
32
33          System.out.println("p1: " + p1.name + ", " + p1.address.
                city); // Output: p1: Alice, Mumbai
34          System.out.println("p2: " + p2.name + ", " + p2.address.
                city); // Output: p2: Bob, Mumbai
35      }
36  }
```

5.5 Deep Copy Example

```
1   class Address implements Cloneable {
2       String city;
3
4       Address(String city) {
5           this.city = city;
6       }
7
8       @Override
9       protected Object clone() throws CloneNotSupportedException {
10          return super.clone();
11      }
12  }
13
14  class Person implements Cloneable {
15      String name;
16      Address address;
17
18      Person(String name, Address address) {
19          this.name = name;
20          this.address = address;
21      }
22
23      @Override
24      protected Object clone() throws CloneNotSupportedException {
25          Person cloned = (Person) super.clone();
```

```
26          cloned.address = (Address) address.clone(); // Deep copy
27          return cloned;
28      }
29  }
30
31  public class Main {
32      public static void main(String[] args) throws
            CloneNotSupportedException {
33          Address addr = new Address("Delhi");
34          Person p1 = new Person("Alice", addr);
35          Person p2 = (Person) p1.clone();
36
37          p2.name = "Bob";
38          p2.address.city = "Mumbai";
39
40          System.out.println("p1: " + p1.name + ", " + p1.address.
                city); // Output: p1: Alice, Delhi
41          System.out.println("p2: " + p2.name + ", " + p2.address.
                city); // Output: p2: Bob, Mumbai
42      }
43  }
```

Important Points:

- Shallow copies share references, leading to unintended changes.
- Deep copies ensure complete independence, critical for complex objects.
- Cloning requires careful implementation to avoid `CloneNotSupportedException`.

6 Practical Tips for Applying These Concepts

6.1 Key Concepts

- **Generics**: Use in collections and custom classes to ensure type safety and reusability.
- **Custom ArrayList**: Implement for interview preparation and to understand collection frameworks.
- **Lambda Expressions**: Use with Streams and functional interfaces for concise, modern code.
- **Exception Handling**: Always handle exceptions gracefully, using custom exceptions for specific cases.
- **Object Cloning**: Use deep copying for objects with nested references to avoid side effects.
- **Real-World Application**: These concepts are critical for frameworks like Spring (generics, lambda expressions) and enterprise systems (exception handling, cloning).

6.2 Example from Video

The video suggests building a `TaskManager` system using generics for tasks, lambda expressions for task processing, and exception handling for error management.

6.3 Code Snippet

```java
import java.util.ArrayList;
import java.util.List;

@FunctionalInterface
interface TaskProcessor {
    void process(String task);
}

class TaskManager<T> {
    private List<T> tasks = new ArrayList<>();

    public void addTask(T task) throws IllegalArgumentException {
        if (task == null) {
            throw new IllegalArgumentException("Task cannot be null");
        }
        tasks.add(task);
    }

    public void processTasks(TaskProcessor processor) {
        for (T task : tasks) {
            processor.process(task.toString());
        }
    }
}

public class Main {
    public static void main(String[] args) {
        TaskManager<String> manager = new TaskManager<>();
        try {
            manager.addTask("Write code");
            manager.addTask("Test app");
            // manager.addTask(null); // Throws IllegalArgumentException
            manager.processTasks(task -> System.out.println("Processing: " + task));
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
// Output:
// Processing: Write code
// Processing: Test app
```

Important Point: This design integrates generics, lambda expressions, and exception handling, demonstrating professional coding practices.

7 Common Pitfalls and How to Avoid Them

7.1 Key Concepts

- **Generics Misuse**: Using raw types (non-generic) can cause runtime errors. Always specify types.
- **Custom ArrayList Bugs**: Failing to resize or handle index bounds can crash the program. Test thoroughly.
- **Lambda Overuse**: Complex lambdas can reduce readability. Use for simple operations or refactor into methods.
- **Exception Swallowing**: Catching `Exception` without handling can hide bugs. Catch specific exceptions.
- **Cloning Errors**: Forgetting to implement `Cloneable` or deep copying references causes issues. Always override `clone()` correctly.

7.2 Example from Video

The video shows a bug where a raw `ArrayList` causes a `ClassCastException`.

7.3 Code Snippet (Bug Example)

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList list = new ArrayList();
        list.add("Hello");
        list.add(42);
        String s = (String) list.get(1); // Runtime error:
            ClassCastException
    }
}
```

Fix with Generics:

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Hello");
        // list.add(42); // Compile-time error
    }
}
```

Important Point: Generics prevent such errors at compile time, ensuring robust code.

8 Connection to Your Goal

To become a top 1% coder by 2027 and land a high-paying tech job in San Francisco, mastering these concepts is essential:

- **Generics**: Widely used in Java collections and frameworks like Spring, critical for scalable systems.
- **Custom ArrayList**: Demonstrates deep understanding of data structures, a common interview topic.
- **Lambda Expressions**: Essential for modern Java (e.g., Streams, reactive programming).
- **Exception Handling**: Ensures robust, production-ready code, a must for enterprise applications.
- **Object Cloning**: Useful for complex object manipulation, relevant in system design.
- **Action Plan**:
  - **Practice**: Build a generic task management system with lambda-based processing and exception handling.
  - **Learn Frameworks**: Explore Spring Boot or Java Streams to apply generics and lambdas.
  - **Coding Challenges**: Solve LeetCode problems using generics and custom data structures.
  - **Portfolio**: Contribute to open-source projects on GitHub, showcasing these concepts in real-world applications.

9 Conclusion

This lecture covers advanced Java concepts critical for professional development. Generics, custom ArrayLists, lambda expressions, exception handling, and object cloning enable you to build robust, scalable systems. Practice these through projects, apply them in real-world scenarios, and master them to align with your goal of becoming an elite coder.

Source: Community Classroom OOP Lecture 6, `https://youtu.be/OY2lPr8h93U`